IT PROJECT REPORT

CipherChat

An encrypted chat that runs in the browser. A room is a link and a password. The server forwards encrypted frames it cannot read and keeps no history.

Okwuchukwu Ekene Don Davies

October 2026

github.com/izumitempest/CipherChat-V4

REPORT

1. Overview

CipherChat is an encrypted chat application I built as an IT project. A room is a link and a password. Anyone with both can join and talk. Messages are encrypted inside the browser before they leave the device, and the server only ever carries encrypted data. The server keeps no message history, so a new member sees nothing from before they joined, and there is nothing on the server to read later.

I designed and built the project end to end: the interface, the browser cryptography, the relay service, the database, the tests, the deployment, and the documentation. This report describes what I built, how the security works, and what the work involved.

The code is public at github.com/izumitempest/CipherChat-V4 under the MIT license, tagged v1.0.0-rc1. The CI pipeline runs green end to end.

2. What I built

The product, in plain terms:

- **Rooms.** A room is created with a generated password and shared by invite link, with up to 12 members. Each member gets a name and a color derived from their signing key, and a verification screen lets members compare key fingerprints to confirm nobody is being impersonated.
- **Encryption everywhere.** Messages, files, typing indicators, quoted replies, and reactions are all encrypted end to end. Every message is signed inside the encrypted payload, so a forged message is detected and quietly rejected.
- **Ephemerality.** Any message can be set to delete itself, from 5 seconds to 24 hours. A room can be given a lifetime from 5 minutes to 30 days, or stay open until someone burns it. Refreshing the page locks every room until the password is entered again, because keys live in memory only.
- **Files that stay inside the app.** Images, video, audio, PDF, text, CSV, and a hex view all open in the app itself. PDFs render on a canvas through pdf.js, so the browser's PDF toolbar, with its save button, is never used. View-once files can be opened exactly once, by one person, and have no download button at all.
- **Phones and desktops.** The app handles the mobile keyboard so it never covers the input, has a dark mode, installs as a PWA, and treats several windows of the same browser in one room as separate members.
- **No accounts, no history, no analytics.**

3. How it works

The system has three parts.

1. The browser holds every secret. The room password is run through argon2id, a memory-hard key derivation function tuned to cost 64 MB of memory and about a second on a normal laptop, to produce the AES-256-GCM key that encrypts the room's traffic. The password never leaves the device, and keys are held in memory only.

2. The server has two services. The web tier, Next.js 16 with Prisma over SQLite, serves the app and a small REST API for room lifecycle and the member registry. The relay, a Bun service using Socket.IO, forwards encrypted frames between members in real time and tracks who is online, in memory. The relay has no database and writes nothing to disk. It is blind by design: it cannot read a single frame it carries.

3. Deployment is one docker compose command. Three containers come up: the web tier, the relay, and Caddy as a reverse proxy, with Caddy obtaining and renewing HTTPS certificates automatically.

Each device keeps one secret seed, stored as a non-extractable key in IndexedDB. Every room derives its own signing keypair from that seed with HKDF, so the same device has a different identity in every room. When a member leaves, the room generates a fresh random 256-bit key, never derived from the password, and delivers it to each remaining member individually, wrapped with ECDH. A former member who still has the password and the link cannot read anything sent after they left.

Area	Choice	Why
Web app	TypeScript, React 19, Next.js 16, Tailwind CSS	one codebase for the app and the API, with end-to-end type checking
Realtime relay	Socket.IO service on Bun	small and fast, and it needs no persistence
Database	Prisma with SQLite	a simple registry of rooms and members, easy to deploy
Cryptography	WebCrypto API: argon2id, AES-256-GCM, ECDSA P-256, HKDF, ECDH	standard primitives, all inside the browser
Testing	Vitest, Playwright	property tests for the security rules, real browsers for full flows
Delivery	Docker Compose, Caddy, GitHub Actions	one-command deployment, CI that runs the real stack

Table 1. Technology choices.

4. Security design

Each security property is enforced by a specific mechanism, and each mechanism is pinned by a test named after it:

Property	How it is enforced
Confidentiality	The password is stretched with argon2id in the browser and messages are encrypted with AES-256-GCM. The server only sees ciphertext of one fixed size.
Authenticity	Every frame is signed with ECDSA P-256 inside the encrypted payload. A frame with a bad signature is rejected.

Property	How it is enforced
Protection after departure	Leaving rotates the room to a fresh random key, delivered pairwise with ECDH. The departed member's old key stops working.
Replay protection	Each sender has a monotonic counter, frames carry timestamps checked against a 10 minute window, frame ids are deduplicated, and counters survive page refreshes.
Traffic analysis resistance	Every frame is padded to one uniform size. A typing indicator, a message, and a burn notice are indistinguishable on the wire.
Wrong password detection	The room stores an encrypted verifier. A joiner who types the wrong password is told so, without the server ever seeing the password.
Password quality	Passwords are six words drawn from a 256-word list with crypto.getRandomValues, about 281 trillion combinations.
Stated limits	The threat model says plainly what this does not protect against, including metadata and endpoint compromise, and ends with the line: if your opponent is a nation-state, use Signal or SimpleX.

Table 2. Security properties and the mechanism behind each one.

5. Security review and fixes

I ran an adversarial review of the whole protocol, reading every message handler with an attacker's mindset. It found real problems. The most serious ones, and what I did about them:

- **Registry poisoning.** The relay accepted join events with no authentication, so anyone could impersonate an offline member and replace their registered key. The member registry is now written only by the REST tier, and a forged join delivers no keys.
- **Unauthenticated departure.** Knowing only the room id, anyone could claim a member had left and force disruptive key rotations. Leaving now requires a valid ECDSA signature from that member's own room key, and the server confirms the member is really gone before rotating.
- **Spoiled view-once files.** A relay event marked view-once files as opened with no verification, so anyone with the room id could ruin them for everyone. That handler was deleted.
- **Replays across refreshes.** Replay state lived only for the page session, so a captured frame could be re-injected into a freshly loaded browser. Replay counters are now saved per room and per device.

Two more rounds of hardening came out of my own testing and questions:

- **Weak password generation.** Room passwords were generated with Math.random, which is not cryptographically secure, over a 24-word list, about 18 bits of entropy. The whole space could be tried against the public verifier in hours on one computer. I rebuilt the generator around crypto.getRandomValues with rejection sampling, a 256-word list, and six words. I also made the password fields uncontrolled inputs, so the password never appears in the DOM value attribute, set autocomplete to new-password so the

browser does not offer to save an ephemeral room password, and cleared each field the moment its job was done.

- **Photo metadata.** Photos can carry EXIF data, including GPS coordinates. I wrote a byte-level scanner for metadata in JPEG, PNG, and WebP, and every image is re-encoded through a canvas before it is sent, which strips the metadata. If an image with metadata cannot be re-encoded, it is refused rather than sent with its data intact.

Two design gaps got their own treatment. A member who just closes the tab never announces a departure, so a silent leaver would keep the key forever: remaining members now watch a two minute grace period, and the server checks live connection state before writing anyone out and re-keying the room. For abuse, a member can end a room at any time, while a stranger who is not in the room needs reports from three different networks before the room is burned. Room existence is never confirmed to strangers either way.

6. Bugs I found and fixed

A few of the more interesting problems, chosen because each needed a real diagnosis rather than a guess:

Problem	Cause	Fix
A third member joining from a second window of the same browser was invisible, and his messages appeared under another member's name	All windows of one browser share the device seed, so every window derived the same room identity and rejoined as the existing member	Additional windows now derive their own per-window identity under a different HKDF label and register as their own members; the first window keeps the original identity
Room passwords were predictable	The generator used Math.random over a 24-word list, about 18 bits	crypto.getRandomValues over a 256-word list, six words, with property tests pinning the randomness source
On phones, the on-screen keyboard covered the chat input and the dialogs	iOS Safari overlays the page with the keyboard instead of resizing the page	A hook measures the visual viewport and publishes the covered height as a CSS variable; the composer and dialogs rise above the keyboard
Every database call failed inside the Docker deployment	The Prisma query engine binary did not match the container's OpenSSL build	openssl is installed in the runtime image and the engine target is pinned in the schema; caught by CI before any real deployment
Real-time connections would have failed behind the reverse proxy	socket.io-client treats a URL path as a namespace, so its requests went to a route the proxy does not serve	The client parses the relay URL itself and passes the path as the transport path; seven tests pin the parsing
First-time visitors got a permanent update notice over their conversation	A service worker flag was changed inside the first-install event, so an install race looked like an update	The flag is captured once at load and never changed; the end-to-end tests now block service workers

Table 3. Selected bugs: the problem, the cause, and the fix.

7. Testing and CI

177	2	3	12
property tests	end-to-end suites	production bugs caught by CI	members per room

- **177 unit and property tests** (Vitest). The security tests are named after the property they protect, for example: a departed member cannot decrypt frames sent after the rotation, and a captured frame that is re-injected is rejected. The rule I held for security work was to write the failing test first, then make it pass.
- **Two Playwright end-to-end suites** run against the real stack. The golden path covers creating a room, joining, messaging, replying, reacting, sending a GPS-tagged photo and byte-verifying on the receiving side that the metadata was stripped, opening a view-once file, leaving, re-keying, rejoining under the new key, and burning the room. The multi-member suite runs three members through a full message matrix and checks that a second window of one browser registers as its own member.
- **CI (GitHub Actions) runs on every push:** lint, type checking, the full test suite, a dependency audit gate, and a production build. A second job then builds the whole Docker stack on the runner and runs the end-to-end suites against it. The first green run came after CI caught three production bugs the development machine could not see; they are in the table above.
- **Releases are automated.** Pushing a version tag runs the suite, and a GitHub release is created only if every test passes. A red suite means no release.

8. Deployment

The goal for deployment was one plain server and no managed services. The compose file starts the web tier, the relay, and Caddy. All URLs and secrets come from environment variables, documented in .env.example, and the internal presence endpoint is guarded by a token that must be rotated on any real deployment. Both containers have healthchecks that fail closed: the web probe is a read-only database round trip, and the relay probe checks a real engine.io handshake rather than just an open port.

Dependency risk is gated as well. The audit blocks the build on high or critical advisories in the runtime dependency tree, and every current exception is written down and justified in AUDIT.md rather than ignored.

9. Documentation

I kept the documentation to the same standard as the code:

- **README.md:** what the app is, how to run it, and what it does not protect against.

- **DESIGN.md:** the design decisions and a full threat model, including exactly what lingers on the device, which is metadata only: room names, creator tokens, and replay counters.
- **COMPONENTS.md and CHANGES.md:** a component reference and a detailed, chronological change record.
- **Terms of Use, a Privacy Policy, SECURITY.md** with a reporting address, **MOBILE.md** for the PWA and app paths, and the MIT license.

The rule I held in every document: any number must survive a check against the code, and known limits are written down rather than hidden.

10. Current state

The repository is public at github.com/izumitempest/CipherChat-V4 with release v1.0.0-rc1 published. CI is green end to end: lint, types, the 177 tests, the audit gate, the production build, and the end-to-end run against the real Docker stack. What remains is operational rather than code: a domain and a server of its own, confirming the abuse mailbox, and a few reviewed tooling advisories documented in AUDIT.md.

11. What the work shows

- Full-stack web development with TypeScript, React, Next.js, Prisma, and Socket.IO, shipped as a working product.
- Applied cryptography in the browser: key derivation, authenticated encryption, signing, key agreement, key rotation, replay defense, and uniform padding, all on standard primitives.
- Secure development habits: adversarial review, failing closed on unexpected input, rate limiting, proof of possession, and secure randomness.
- Testing that earns its keep: property-driven security tests, browser end-to-end suites, and a CI pipeline that catches what the development machine cannot.
- Deployment and operations with Docker, Caddy, and GitHub Actions, including automated, test-gated releases.
- Technical writing that states what is true, including the limits.

Okwuchukwu Ekene Don Davies, October 2026